# EE5203 Lab 5 Guide

*Button interrupts: replace polling with EXTI - Basri Erdogan*

**Mission:** Configure B1 (PC13) as a falling-edge interrupt so that one valid press toggles the LED on PA5 — without reading the button anywhere in `while (1)`.

This lab uses Lecture 6 (EXTI, NVIC, callbacks, `volatile` flags, debounce).

## Prepare before class

- ☐ Bring your board and a USB **data** cable.
- ☐ Do the **Lab 5 pre-lab quiz** on CATS.
- ☐ Re-read the theory slides up to "The complete interrupt path with register names".
- ☐ Know why pressing B1 produces a *falling* edge on PC13.

## Learning objectives

By checkoff time, you can:

1. configure an EXTI pin and enable its NVIC request in CubeMX;
2. prove with a breakpoint that the interrupt path reaches your callback;
3. pass an event from the callback to `main()` with a `volatile` flag;
4. debounce the button so one press produces exactly one event.

## Hardware and files

| Item | Value |
|---|---|
| Board | Nucleo-F401RE: LED on PA5, B1 on PC13 (external pull-up, **low while pressed**), no wiring |
| Tools | STM32CubeMX (new project) + VS Code |
| Project name | `EE5203_L05_<studentnumber>` |
| File you edit | `main.c`, only inside the `USER CODE` regions |

> **i** Using your own laptop? (click)
>
> Follow the steps as written, with these changes. Set up your laptop once with the Home Setup Guide.
> - **Folder:** `C:\STM32\EE5203` (Mac: `~/STM32/EE5203`), not `C:\Work`.
> - **Step 5:** skip it. Open the folder, choose **Debug**, then ST icon → **Set Up STM32Cube project(s)**.
> - **Build / debug: F7** (Mac: **fn + F7**), then Run and Debug → **STM32Cube: STM32 Launch ST-Link GDB Server**.
> - **End of session:** no *Save to H*. Your project stays on your laptop.

## Checkpoint A - New EXTI project and callback proof

**Start of the session:** if `C:\Work\<studentnumber>` does not contain your Lab 4 project, copy **all** your project folders from `H:\EE_Labs` into it first.

1. **STM32CubeMX:** Board Selector → **NUCLEO-F401RE** → accept the default initialization. Set up **PA5** as in Lab 4: **Reset_State**, then `GPIO_Output`, User Label empty.
2. Keep **PC13** as `GPIO_EXTI13`. In System Core → GPIO, PC13: **External Interrupt Mode with Falling edge trigger detection**, **No pull-up and no pull-down** (the board's resistor already holds PC13 high).
3. In System Core → **NVIC**: enable **EXTI line[15:10] interrupts** (PC13 is line 13; lines 10–15 share this entry). Leave the priority at its default.
4. Project Manager: name `EE5203_L05_<studentnumber>`, location `C:\Work\<studentnumber>`, Toolchain/IDE = **CMake** → Generate Code.
5. Copy `C:\Users\Public\EE5203_template\.vscode` into the project folder.
6. Check the generated setup in `MX_GPIO_Init()`:

```
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

7. Add below `main()`, in the user-code section for functions:

```c
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}
```

8. Build (**Ctrl+Shift+B**), start the debugger (**F5**), Continue, press B1: the LED responds without any button code in the loop.

**Show:** a breakpoint on the `if` line is hit when you press B1, and `GPIO_Pin` equals `GPIO_PIN_13`.

## Checkpoint B - The callback reports, the loop acts

The LED action does not belong in the callback. In `USER CODE BEGIN PV`:

```c
volatile uint8_t button_event = 0;
```

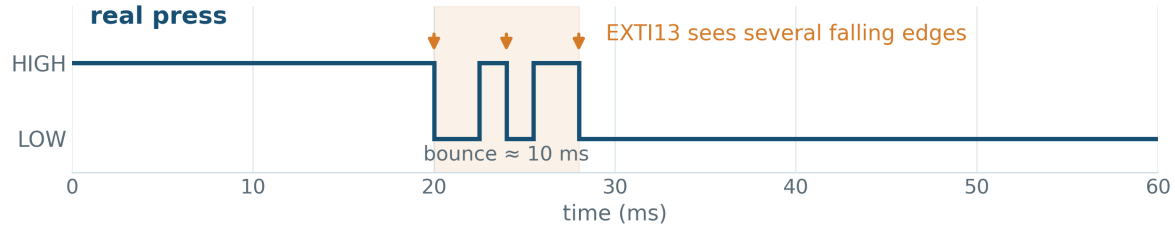
Change the callback body to only record the event (`button_event = 1;`), and let the main loop consume it in `USER CODE BEGIN 3`:

```c
if (button_event == 1) {
    button_event = 0;               /* consume before acting */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
}
```

**Show:** the callback with one statement, and the LED toggle in `main()`.

## Checkpoint C - One press, one event

A real contact bounces — several falling edges per press:




Add `uint32_t last_press_ms = 0;` beside the flag and accept a press only when at least 80 ms has passed since the last accepted one:

```c
uint32_t now = HAL_GetTick();
if ((now - last_press_ms) >= 80) {
    last_press_ms = now;
    button_event = 1;
}
```

The subtraction stays correct even when the millisecond counter wraps.

**Check it:** test behavior, not just compilation. Record unexpected behavior *before* changing the code.

| Test | Expected observation |
|---|---|
| press once slowly | LED toggles once |
| press ten times | LED toggles ten times |
| hold B1 down | no repeated toggling |
| release B1 | no toggle |

**Show:** the four tests in the table.

## Checkpoint D - Two valid presses per toggle

No code is given: toggle the LED only after **two** valid presses, using a counter in the main loop; the interrupt and debounce code stay unchanged.

**Show:** four presses give exactly two toggles; explain where your counter lives.

## Optional (not graded) - Two-state LED control

After Checkpoint D: two states, `STEADY` and `BLINK` (the LED toggles every 250 ms); each valid press switches the state. Time it with `HAL_GetTick()`, not `HAL_Delay()`.

## Common problems

Show the table (click)

Fix the **first** failed stage along the path B1/PC13 → EXTI13 → `EXTI15_10_IRQn` → handler → callback → `button_event` → main loop.

| What you see | First check |
|---|---|
| The callback breakpoint is never hit | PC13 is `GPIO_EXTI13` and **EXTI line[15:10]** is enabled in NVIC; you generated code again. |
| The callback runs, the LED does not change | Inspect `GPIO_Pin`: compare with `GPIO_PIN_13`. |
| The LED toggles on **release** | Rising edge selected: choose falling edge in CubeMX. |
| The flag is set, `main()` never sees it | `button_event` is not declared `volatile`. |
| Several toggles per press | The 80 ms guard or `last_press_ms = now;` is missing. |
| The LED never lights | PA5 is `GPIO_Output` in the `.ioc`. |
| My code disappeared after Generate Code | It was outside `USER CODE`. Paste it again inside the region. |
| `...FLASH.ld: syntax error` | Build with **Ctrl+Shift+B**; check that `.vscode` is in the project folder. |
| `...FLASH.ld: syntax error` on your own laptop | Replace `FLASH.ld` with the file from the Home Setup Guide (step 6). Repeat after every Generate Code. |

## End of the session - Save to H

Show the steps (click)

Close VS Code, run the **Save to H** desktop shortcut, and check that `H:\EE_Labs\EE5203_L05_<studentnumber>` exists before you log out.

## Acceptance checklist

- ☐ `.ioc`: PC13 = `GPIO_EXTI13`, falling edge, no pull, NVIC enabled; code only inside `USER CODE`.
- ☐ Checkpoint A: breakpoint proves `HAL_GPIO_EXTI_Callback` runs on a press.
- ☐ Checkpoint B: `button_event` is `volatile`; the LED action lives in `main()`.
- ☐ Checkpoint C: the four tests pass, one toggle per press.
- ☐ Checkpoint D: two valid presses per toggle.

## Individual oral check

Be ready for **one** question **or** one live change, chosen by your lab assistant. For a live change, predict the result first, then make the change.

Questions:

- Why does B1 use a falling edge and no internal pull?
- What runs first: `EXTI15_10_IRQHandler()` or your callback?
- Why must the callback stay short, and why is `volatile` required?
- What would happen without the 80 ms guard?

Live changes:

- Set PC13 to the rising edge in CubeMX. When does the LED toggle now?
- Turn off **EXTI line[15:10]** in NVIC. Does the callback still run?
- Change the guard from 80 ms to 1000 ms and press B1 quickly several times.

## Submit

Submit `main.c`, the `.ioc` file, and one debugger screenshot showing the breakpoint inside `HAL_GPIO_EXTI_Callback`.

**Marks (10):** pre-lab quiz on CATS 1 · A project and callback proof 2 · B event flag 2 · C debounce tests 2 · D two presses per toggle 1 · oral check (one question or one live change) 2. The two-state LED control is optional and not graded.